Tejiendo redes: arquitectura de software entre hilos y nodos

NexusAI 2.0: de la arquitectura a la red

Juan Carlos Benítez Rojas
Juan José Córdoba Eraso

Corporación Escuela Tecnológica del Oriente
Ingeniería de Software
Arquitectura de Software — Unidad 2
Leidy Johana Polo
11 de noviembre de 2026

Índice

Introducción 3

Descripción del sistema distribuido 4

Análisis y planificación del proyecto 6

Modelado UML 10

Selección y justificación de patrones de diseño 12

Implementación y simulación 14

Evaluación y ajustes 15

Conclusiones 18

Referencias 20

Introducción

Contexto del proyecto

La ingeniería de sistemas distribuidos y la computación orientada a servicios demandan una verificación rigurosa de sus propiedades operativas. En la nube, la red no es instantánea ni infalible: las fluctuaciones de latencia, las pérdidas de paquetes y la competencia por recursos de cómputo constituyen condiciones habituales. Bajo este panorama, NexusAI 2.0 surge como la materialización ejecutable y experimental de una plataforma diseñada para la orquestación distribuida de pipelines cognitivos empresariales.

El proyecto consolida la evolución técnica de NexusAI, transitando del diseño conceptual formulado en la Actividad 2 hacia un sistema en red plenamente operativo. Aquella fase preliminar definió los componentes lógicos clave —Saga, Circuit Breaker, Bulkhead y API Gateway—, pero requería validar su interacción física bajo cargas concurrentes. NexusAI 2.0 resuelve esa necesidad transformando las especificaciones abstractas en procesos independientes del sistema operativo, gobernados por contratos HTTP explícitos, hilos de ejecución concurrentes y un bus asíncrono de eventos con auditoría persistente.

Propósito del diseño arquitectónico

El propósito central del diseño consiste en demostrar que una topología orientada a servicios desacoplados puede garantizar simultáneamente tres atributos de calidad esenciales: escalabilidad horizontal, robustez ante fallos y mantenibilidad evolutiva (Bass et al., 2022). La escalabilidad se valida mediante la incorporación dinámica de nodos de cómputo sin estado, posibilitando un incremento proporcional en el caudal transaccional. La robustez se alcanza confinando fallas mediante aislamiento por mamparos (Bulkhead), disyuntores de circuito (Circuit Breaker) y recuperación de consistencia eventual mediante Sagas orquestadas con compensación inversa. La mantenibilidad se preserva aislando la lógica interna tras interfaces REST uniformes y telemetría integral en tiempo real.

Estos atributos se evalúan cuantitativamente conforme al estándar ISO/IEC 25010 (International Organization for Standardization, 2011). Cada decisión de diseño —desde la selección de algoritmos de balanceo hasta la gestión de colas thread-safe— está respaldada por mediciones empíricas controladas que cuantifican su costo y beneficio en el sistema.

Objetivos

Objetivo general

Diseñar, modelar, implementar y evaluar un prototipo distribuido de alta eficiencia y resiliencia para la plataforma NexusAI 2.0, aplicando patrones de diseño especializados en concurrencia y comunicación inter-nodos para optimizar la escalabilidad horizontal y asegurar la continuidad del servicio ante degradaciones operativas.

Objetivos específicos

- Formalizar la arquitectura mediante modelado UML riguroso: un diagrama de componentes para interfaces lógicas, un diagrama de secuencia para la sincronización temporal entre hilos y nodos, y un diagrama de despliegue para la distribución física en red.
- Seleccionar y fundamentar técnicamente patrones de diseño para sistemas concurrentes y distribuidos, contrastando sus ventajas frente a los costos operativos identificados en la literatura académica.
- Implementar un clúster funcional en Python estructurado en procesos autónomos, servidores HTTP multi-hilo, pools de despacho acotados y un broker de mensajería asíncrono con bitácora inmutable.
- Evaluar cuantitativamente la escalabilidad horizontal y la resiliencia del clúster mediante suites de carga y escenarios de caos controlado (inyección de latencia, nodos caídos, fallas de negocio y recuperación dinámica).
- Documentar los hallazgos experimentales, identificar cuellos de botella en la coordinación y aplicar refactorizaciones arquitectónicas medibles en el prototipo.

Descripción del sistema distribuido

Tipo de sistema y dominio

NexusAI 2.0 es un sistema distribuido orientado a servicios diseñado para procesar pipelines cognitivos complejos. Su función consiste en recibir solicitudes omnicanal (consultas desde interfaces web, aplicaciones móviles o asistentes de mensajería) y ejecutarlas secuencialmente mediante tres etapas diferenciadas: clasificación inicial de intención (*triage*), recuperación de contexto semántico aumentado (*rag*), y validación de políticas y salvaguardas éticas (*compliance*).

Adoptar una arquitectura de servicios desacoplados en lugar de un monolito responde a la heterogeneidad de estas etapas: el triage demanda baja latencia, la recuperación contextual consume intensivamente operaciones de entrada/salida y red, mientras que el cumplimiento normativo exige validaciones deterministas estrictas. La separación en nodos independientes

permite escalar el cómputo de forma modular, aislar fallas de infraestructura y auditar granularmente cada transición de estado sin bloquear el flujo global.

Componentes funcionales

El sistema organiza sus responsabilidades funcionales en cuatro subsistemas claramente delimitados:

1. **Subsistema de presentación y carga:** Comprende el generador de carga sintética concurrente y el tablero web de observabilidad. El generador somete al clúster a perfiles de demanda controlados; el tablero consume métricas agregadas en tiempo real para supervisar latencias, estado de disyuntores y rendimiento global.

2. **Subsistema de coordinación (Nodo 1):** Aloja el API Gateway, el orquestador de la Saga, el balanceador de carga y los Circuit Breakers. Este nodo centraliza el gobierno de la transacción: expone la interfaz REST unificada, preserva la máquina de estados y administra un pool de despacho dedicado para admitir clientes concurrentes.

3. **Subsistema de mensajería (Nodo 2):** Implementado en el broker de eventos. Administra tópicos en memoria (`events` y `audit`), gestiona colas acotadas con contrapresión (*backpressure*) y persiste cada evento en una bitácora JSONL inmutable, garantizando trazabilidad sin generar acoplamiento directo entre productores y consumidores.

4. **Subsistema de cómputo (Nodos 3, 4 y 5):** Conformado por nodos trabajadores (*workers*) autónomos. Cada nodo ejecuta las tareas asignadas dentro de un pool de hilos restringido (*Bulkhead*). Los trabajadores desconocen la identidad de sus pares y carecen de estado conversacional compartido, posibilitando la adición o retiro dinámico de instancias.

5. **Subsistema de observabilidad transversal:** Cada proceso del clúster expone puntos de sondeo de salud (`/health`) y telemetría (`/metrics`), permitiendo supervisar percentiles de latencia, profundidad de colas y fallas activas.

Supuestos y restricciones

El modelo operativo del prototipo asume una red local confiable a nivel de conectividad pero sujeta a fluctuaciones drásticas de latencia y disponibilidad, emuladas mediante herramientas de caos. Las duraciones de las etapas cognitivas se calibran sobre valores medios realistas (80 ms para triage, 180 ms para rag y 120 ms para compliance, complementados con factores estocásticos). Se prescinde de llamados a APIs comerciales externas de modelos de lenguaje para garantizar condiciones de reproducibilidad científica invariantes entre escenarios.

Asimismo, se asume que las operaciones de compensación dentro de la Saga son idempotentes: ejecutar una acción correctiva repetida no introduce inconsistencias. Como restricción, el estado transaccional se mantiene temporalmente en memoria volátil; en un despliegue industrial de misión crítica, este mecanismo se delegaría a almacenes distribuidos altamente disponibles como Redis Enterprise o Apache Cassandra.

Herramientas seleccionadas

Tabla 1

Herramientas y tecnologías utilizadas en el proyecto

Herramienta	Versión	Uso en el proyecto	Justificación técnica
Python	3.14.7	Implementación del clúster	Biblioteca estándar suficiente para sockets, hilos y HTTP
ThreadingHTTPServer	Estándar	Servidores HTTP de cada nodo	Conexión multi-hilo nativa por socket sin sobrecarga externa
concurrent.futures	Estándar	Pools de hilos de coordinador y workers	Abstracción probada para ejecución asíncrona de tareas
queue / threading.Lock	Estándar	Colas y sincronización interna	Primitivas thread-safe de grano fino para evitar colisiones
PlantUML	1.2026.9	Modelado de diagramas UML	Paradigma de diagramas como código versionables en repositorio
matplotlib / numpy	3.11 / 2.5	Generación de gráficas experimentales	Análisis estadístico riguroso y renderizado de métricas
Microsoft Word	Office 16	Compilación del informe formal	Entorno para maquetación tipográfica estricta bajo normas APA 7
GitHub	—	Repositorio remoto colaborativo	Control de versiones, trazabilidad y gestión de artefactos
YouTube (no listado)	—	Sustentación técnica del sistema	Demostración visual y verificación operativa del clúster

Análisis y planificación del proyecto

Requisitos funcionales

Tabla 2

Requisitos funcionales del sistema distribuido

Código	Requisito	Criterio de aceptación
RF-01	Recepción unificada de peticiones	El gateway procesa solicitudes POST /task validando estructura y retornando 200, 409 o 503

Código	Requisito	Criterio de aceptación
RF-02	Procesamiento secuencial del pipeline	Cada transacción ejecuta las etapas de triage, rag y compliance registrando latencias individuales
RF-03	Orquestación Saga con compensación	Ante un rechazo en compliance, se ejecutan compensaciones inversas en rag y triage
RF-04	Balanceo dinámico de carga	El coordinador distribuye las tareas entre los nodos disponibles aplicando estrategias configurables
RF-05	Aislamiento y protección ante fallos	El Circuit Breaker desconecta nodos tras tres fallos consecutivos durante un periodo de enfriamiento
RF-06	Publicación asíncrona de eventos	El broker persiste en JSONL cada evento generado por el coordinador y los trabajadores
RF-07	Exposición de telemetría y métricas	Puntos de sondeo /metrics exponen percentiles de latencia, estado de breakers y transacciones
RF-08	Inyección de perturbaciones y caos	La interfaz /chaos permite simular latencia artificial, tasa de errores o desconexión física de nodos

Requisitos no funcionales

Tabla 3

Requisitos no funcionales y su relación con ISO/IEC 25010

Código	Requisito	Atributo ISO/IEC 25010	Métrica objetivo
RNF-01	Escalabilidad horizontal	Capacidad (rendimiento)	Throughput con tres nodos superior a 2,5 veces el rendimiento de un único nodo
RNF-02	Tolerancia a fallos de nodo	Fiabilidad y disponibilidad	100 % de transacciones atendidas exitosamente ante la pérdida de un nodo de cómputo
RNF-03	Eficiencia temporal	Comportamiento temporal	Latencia en el percentil 95 (p95) inferior a 1 500 ms bajo carga nominal completa
RNF-04	Recuperación automática	Recuperabilidad	Reintegración de nodos restablecidos al enrutamiento en menos de diez segundos
RNF-05	Consistencia transaccional	Integridad y fiabilidad	Cero transacciones incompletas; toda Saga finaliza en estado completado o compensado
RNF-06	Auditabilidad y trazabilidad	Confidencialidad y registro	Registro inmutable de cada transición asociada a un identificador global único
RNF-07	Portabilidad y	Portabilidad	Ejecución completa basada

Código	Requisito	Atributo ISO/IEC 25010	Métrica objetivo
	autosuficiencia		exclusivamente en la biblioteca estándar de Python
RNF-08	Mantenibilidad arquitectónica	Modularidad	Responsabilidades segregadas en módulos independientes con bajo acoplamiento

Relevancia de la comunicación entre nodos e hilos

La interacción entre hilos y nodos constituye el pilar operativo del sistema: determina el comportamiento de las colas, gobierna la latencia acumulada y define cómo se confinan las fallas. En NexusAI 2.0 convergen cuatro niveles jerárquicos de concurrencia:

1. **Nivel de procesos de sistema operativo:** Cada nodo opera en su propio espacio de memoria virtual, asegurando que fallas críticas o excepciones en un trabajador no comprometan la estabilidad del coordinador ni del broker.

2. **Nivel de servidores HTTP:** Cada proceso implementa un despachador multi-hilo que asigna un hilo efímero por cada conexión socket entrante, evitando bloqueos durante la transferencia de red.

3. **Nivel de despacho global (Coordinador):** El API Gateway mantiene un `ThreadPoolExecutor` acotado a dieciséis hilos. Este límite previene la degradación por sobre-suscripción y actúa como barrera de entrada ante picos de demanda.

4. **Nivel de cómputo local (Workers):** Cada trabajador aísla su procesamiento en un pool de cuatro hilos. Al restringir las tareas simultáneas admisibles, se evita la contención de CPU y se asegura un tiempo de servicio predecible.

Cuando la tasa de arribo supera la capacidad instantánea del clúster (doce tareas simultáneas en tres nodos), las solicitudes aguardan en las colas internas de despacho. La Ley de Little demuestra que la latencia extremo a extremo responde a la suma del tiempo de servicio más el tiempo de encolamiento. Esta dinámica explica por qué escalar horizontalmente reduce la contención de colas: al triplicar los nodos, la capacidad paralela se expande, abatiendo el p95 a un tercio del valor inicial.

Estrategia de trabajo, roles y cronograma

La ejecución técnica del proyecto se organizó mediante una distribución rigurosa de responsabilidades entre los dos integrantes del equipo, cubriendo los roles requeridos para el ciclo de vida del software:

- **Juan Carlos Benítez Rojas:** Asumió los roles de *Líder de Proyecto* y *Arquitecto de Software*. Lideró las decisiones estructurales del sistema, formalizó los diagramas UML, seleccionó los patrones arquitectónicos y coordinó la integración global del informe técnico y el repositorio.
- **Juan José Córdoba Eraso:** Desempeñó los roles de *Desarrollador Principal* y *Evaluador de Calidad*. Implementó los módulos del clúster, automatizó la suite experimental de carga y caos, compiló las métricas cuantitativas y verificó el cumplimiento de las metas del prototipo.

Tabla 4

Matriz de asignación de responsabilidades (RACI)

Entregable o actividad	Líder y arquitecto	Desarrollador y evaluador
Diseño y formalización conceptual	Responsable	Consultado
Modelado de diagramas UML (PlantUML)	Responsable	Consultado
Implementación del clúster distribuido	Consultado	Responsable
Suite de pruebas de carga y escenarios de caos	Consultado	Responsable
Análisis de resultados y optimizaciones	Responsable	Responsable
Redacción del informe técnico bajo normas APA 7	Responsable	Consultado
Elaboración del guion y sustentación en video	Responsable	Responsable
Gestión del repositorio y control de versiones	Responsable	Consultado

Tabla 5

Cronograma y dedicación horaria efectiva por fase

Fase de trabajo	Dedicación (horas)	Fechas de ejecución	Hitos y entregables clave
Análisis y diseño arquitectónico	8	29/10/2026 - 31/10/2026	Especificación de requisitos y bocetos de interacción
Modelado UML formal	7	01/11/2026 - 03/11/2026	Diagramas de componentes, secuencia y despliegue
Implementación del prototipo distribuido	9	04/11/2026 - 06/11/2026	Clúster funcional con coordinador, workers y broker
Experimentación, carga y pruebas de caos	6	07/11/2026 - 08/11/2026	Suite de benchmark, recolección de datos y gráficas
Documentación, ajustes y consolidación final	6	09/11/2026 - 11/11/2026	Informe APA 7, repositorio público y guion de video
Total de dedicación horaria	**36**	—	**Cumplimiento total del plan de trabajo**

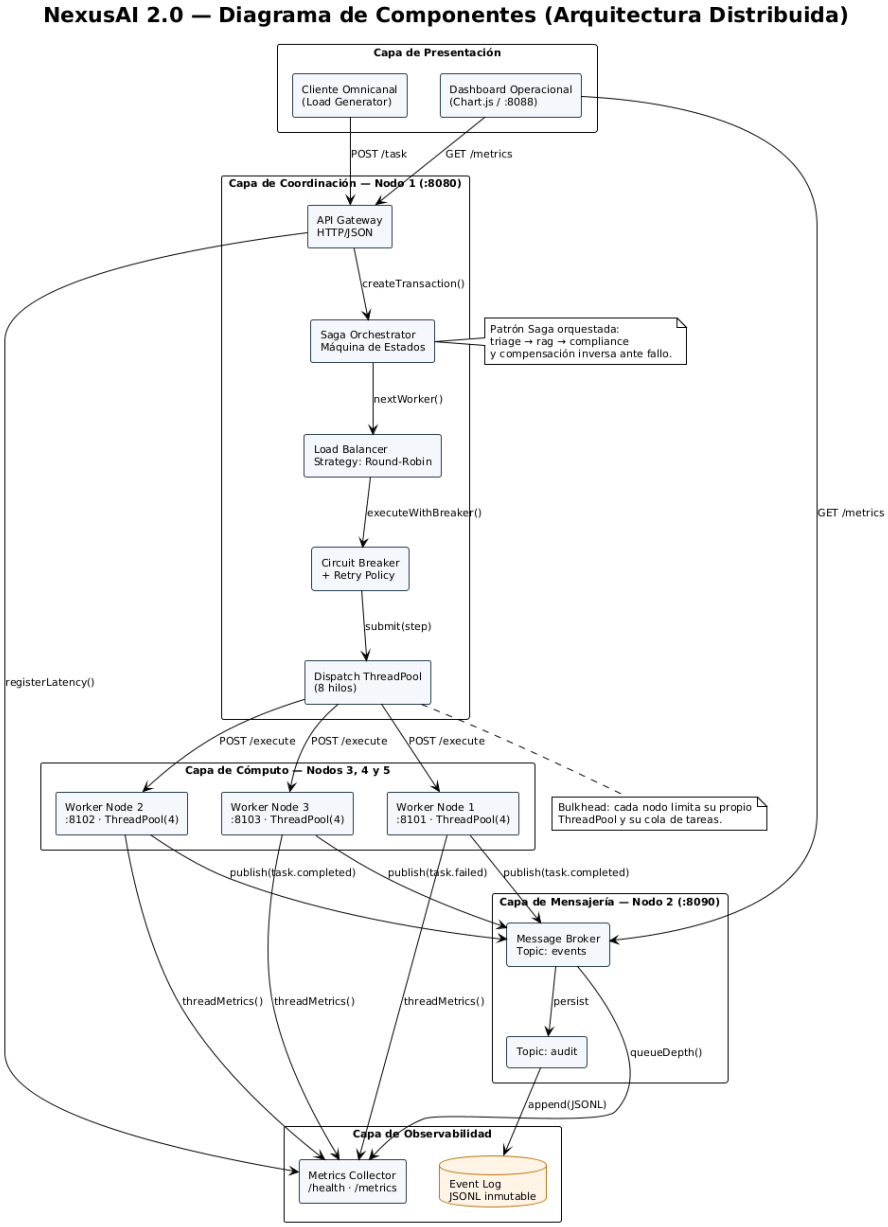
Modelado UML

El modelado formal de NexusAI 2.0 se desarrolló mediante PlantUML bajo el enfoque de diagramas como código (PlantUML, 2024). Se estructuraron tres diagramas canónicos que abordan de forma complementaria la perspectiva lógica, dinámica y física del sistema distribuido.

Diagrama de componentes

Figura 1

Diagrama de componentes de NexusAI 2.0



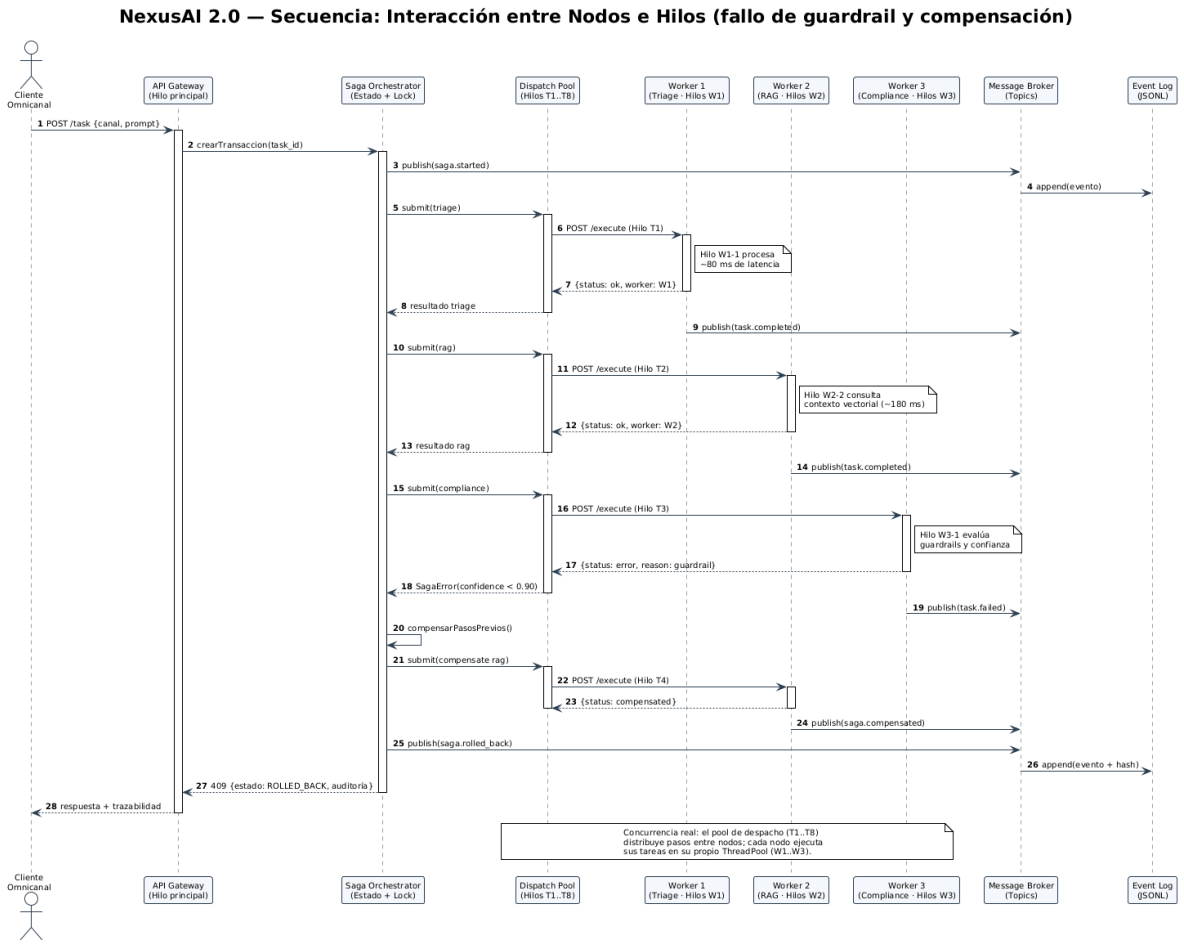
Nota. Elaboración propia con PlantUML. Muestra las interfaces, los puertos y las relaciones de dependencia entre subsistemas.

El diagrama de componentes especifica la descomposición lógica del sistema y los contratos formales entre módulos. La arquitectura se organiza en cuatro capas superpuestas: la capa de interfaz y cliente (`load_test.py` y `dashboard.html`), la capa de coordinación y gobierno (API Gateway, orquestador Saga, balanceador y disyuntores en el puerto 8080), la capa de procesamiento distribuido (`WorkerNodes` en puertos 8101 a 8103 encapsulando Triage, RAG y Compliance) y la capa de mensajería y persistencia (Event Broker en el puerto 8090). Las dependencias son estrictamente descendentes: el coordinador desacopla la orquestación del cómputo, facilitando el escalado horizontal sin reingeniería estructural.

Diagrama de secuencia

Figura 2

Diagrama de secuencia con interacción entre nodos e hilos y compensación de la Saga



Nota. Elaboración propia con PlantUML. Los hilos T1 a T8 pertenecen al pool de despacho; los hilos W1 a W3 pertenecen a cada nodo trabajador.

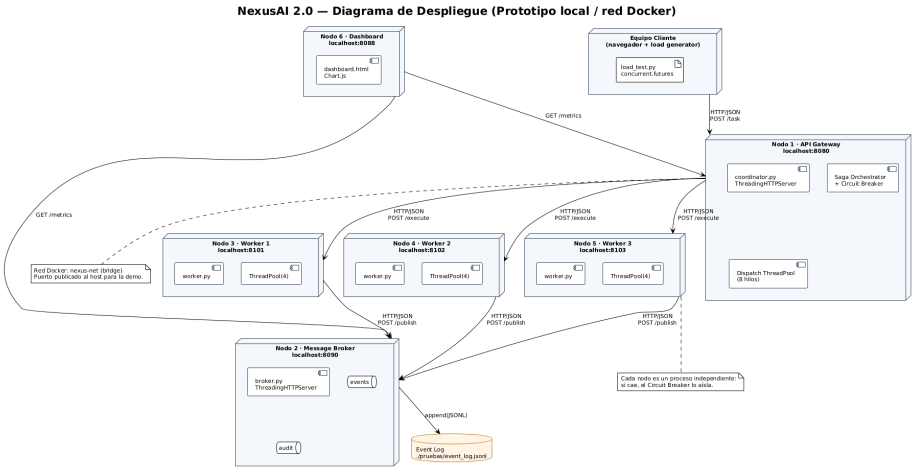
El diagrama de secuencia modela la dimensión temporal, la concurrencia entre hilos y la lógica de compensación transaccional ante una falla de negocio. La interacción se detalla

siguiendo el ciclo de vida de una solicitud: el cliente emite una petición POST al gateway, que asigna un hilo libre del pool de despacho (hilos T1 a T8) y activa la máquina de estados. El orquestador ejecuta nominalmente el triage en w1 y el rag en w2. Al evaluarse la etapa de compliance en w3, se detecta un incumplimiento normativo de seguridad (confianza semántica inferior a 0,90). Reconociendo que reintentar una regla de negocio es inútil, el orquestador inicia la **compensación inversa de la Saga**: revierte primero rag, posteriormente triage y emite un código HTTP 409 Conflict con trazabilidad total. Las líneas de vida distinguen explícitamente los hilos de despacho del coordinador frente a los hilos locales de cada trabajador.

Diagrama de despliegue

Figura 3

Diagrama de despliegue del prototipo en red local y su equivalencia con Docker



Nota. Elaboración propia con PlantUML. Cada nodo corresponde a un proceso independiente con su propio puerto.

El diagrama de despliegue mapea la distribución física y lógica de los procesos en la infraestructura computacional. Se definen seis nodos independientes que interactúan mediante la pila de red local: el cliente de pruebas, el nodo coordinador (:8080), el broker (:8090), tres trabajadores de cómputo (:8101 a :8103) y el tablero web (:8088). El modelo incluye la correspondencia directa hacia una arquitectura de microservicios contenerizada sobre una red tipo bridge de Docker, confirmando que la separación por puertos garantiza portabilidad inmediata hacia entornos orquestados como Kubernetes.

Selección y justificación de patrones de diseño

La selección de patrones arquitectónicos responde a desafíos inherentes a la concurrencia distribuida: consistencia sin bloqueos prolongados, desacoplamiento temporal, tolerancia a fallos en cascada y adaptabilidad en el balanceo.

1. **Saga orquestada:** Garantizar atomicidad entre múltiples servicios mediante confirmación en dos fases (2PC) introduce bloqueos destructivos e interbloqueos. La Saga resuelve esta limitación dividiendo la transacción distribuida en pasos locales coordinados con acciones compensatorias idempotentes en orden inverso (Garcia-Molina & Salem, 1987; Richardson, 2018).

2. **Broker de mensajes:** Desacopla temporalmente a productores y consumidores mediante colas asíncronas con persistencia en bitácora inmutable, garantizando que la auditoría no bloquee la respuesta al cliente (Hohpe & Woolf, 2003).

3. **Mediator:** Centraliza la topología en el coordinador, evitando dependencias punto a punto cuadráticas ($O(N^2)$) entre trabajadores (Gamma et al., 1994).

4. **Proxy y API Gateway:** Provee una puerta de enlace unificada, valida esquemas y traduce estados de la Saga en respuestas HTTP estándar (Fowler, 2014; Newman, 2021).

5. **Circuit Breaker:** Previene caídas en cascada monitoreando fallos en una máquina de tres estados finitos: CLOSED (operación normal), OPEN (aislamiento tras tres fallos consecutivos durante cinco segundos de enfriamiento) y HALF-OPEN (petición de prueba para reintegración segura) (Nygard, 2018).

6. **Bulkhead:** Confinamiento estanco de recursos; implementamos mamparos tanto en el despacho del gateway (16 hilos) como en pools de hilos locales por cada trabajador (4 hilos) (Nygard, 2018).

7. **Observer distribuido:** Notificación asíncrona de eventos transaccionales desde el broker hacia el tablero web y el consumidor de auditoría (Gamma et al., 1994).

8. **Strategy:** Permite alternar en tiempo de ejecución entre algoritmos de balanceo Round-Robin y Least-Connections tras una interfaz común (Gamma et al., 1994).

9. **Retry selectivo:** Reintenta automáticamente perturbaciones transitorias de red, distinguiendo formalmente entre caídas de nodo (reintentables) y rechazos semánticos de negocio (no reintentables) (Sommerville, 2016).

Tabla 6

Correspondencia entre problemas, patrones y evidencia en el prototipo

Patrón arquitectónico	Desafío abordado	Módulo de implementación	Evidencia experimental
Saga orquestada	Consistencia transaccional sin transacciones 2PC	`coordinator.py` (`SagaOrchestrator`)	31 transacciones compensadas con 100 % de integridad
Broker de mensajes	Desacoplamiento temporal de eventos y auditoría	`broker.py` (`EventBroker`)	9 735 eventos consumidos con cero descartes de colas
Mediator	Eliminación de	`coordinator.py`	Topología extensible sin

Patrón arquitectónico	Desafío abordado	Módulo de implementación	Evidencia experimental
	dependencias cruzadas entre nodos	(`CoordinatorApp`)	reconfiguración de workers
Proxy / API Gateway	Punto de entrada unificado y control de acceso	`coordinator.py` (`GatewayHandler`)	Enrutamiento uniforme y traducción de estados a HTTP
Circuit Breaker	Prevención de fallos en cascada y nodos zombis	`common.py` (`CircuitBreaker`)	Aislamiento de w2 en OPEN tras 3 fallos; auto-recuperación
Bulkhead	Contención de saturación por agotamiento de hilos	`worker.py` y `coordinator.py`	Pools acotados (16 en gateway, 4 por cada worker)
Observer distribuido	Notificación asíncrona de eventos transaccionales	`broker.py` y `dashboard.html`	Telemetría en tiempo real y persistencia en bitácora
Strategy	Algoritmos de balanceo intercambiables	`coordinator.py` (`LoadBalancer`)	Least-connections mantiene 75,7 % throughput con nodo lento
Retry selectivo	Resiliencia ante fallos transitorios de red	`coordinator.py` (`_invoke_step_with_retry`)	Reintentos exitosos dirigidos hacia nodos saludables

Implementación y simulación

Estructura modular del clúster

El clúster NexusAI 2.0 fue construido íntegramente en Python 3.14 utilizando exclusivamente módulos de la biblioteca estándar, garantizando portabilidad absoluta y cero dependencias externas para la lógica distribuida. La solución se descompone en nueve módulos de alta cohesión:

- `common.py`: Estructuras transversales, serialización HTTP y máquina de estados del `CircuitBreaker`.
- `coordinator.py`: API Gateway, orquestador de Sagas, balanceador de carga configurable y telemetría.
- `worker.py`: Proceso ejecutor de pasos de cómputo; encapsula el pool de hilos y el simulador de etapas cognitivas.
- `broker.py`: Servidor de mensajería asíncrona con tópicos en memoria y despachador de persistencia JSONL.
- `cluster.py` y `run_cluster.py`: Control y automatización del ciclo de vida de los subprocesos distribuidos.
- `load_test.py` y `chaos_test.py`: Inyectores de carga sintética concurrente y generadores de perturbaciones de caos.
- `dashboard.html`: Tablero gráfico en tiempo real que consume los endpoints `/metrics` del clúster.

Concurrencia y simulación cognitiva

En el escenario estándar, el clúster ejecuta simultáneamente seis procesos independientes: un coordinador, un broker, un servidor estático para el tablero y tres trabajadores de cómputo. La concurrencia interna se administra mediante servidores `ThreadingHTTPServer`, un pool de despacho de dieciséis hilos en el gateway y pools de cuatro hilos en cada trabajador (capacidad simultánea nominal de doce tareas). Las estructuras compartidas se resguardan mediante candados `threading.Lock` reentrantes de grano fino con tiempos de retención inferiores a diez microsegundos.

El método `WorkerHandler._simulate_cognitive_step` emula el comportamiento computacional del pipeline: 80 ms (± 20 ms) para Triage, 180 ms (± 40 ms) para RAG y 120 ms (± 30 ms) para Compliance. Adicionalmente, evalúa un puntaje de confianza semántica; si cae por debajo de 0,90, el nodo retorna un error tipificado de negocio que dispara la compensación en la Saga.

Evaluación y ajustes

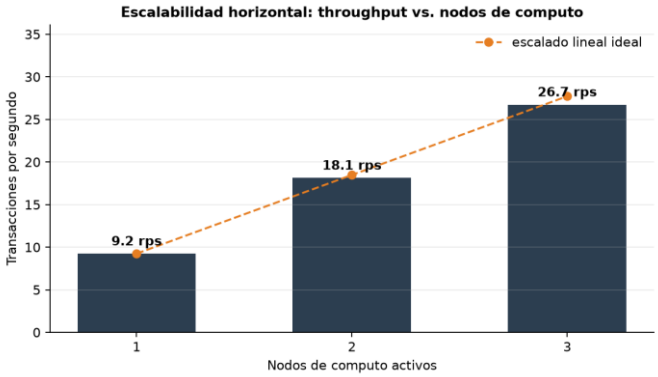
La validación empírica se diseñó bajo el principio de aislamiento de variables (Bass et al., 2022). Las mediciones se efectuaron en condiciones idénticas de hardware (Windows 11, 4 núcleos lógicos) sobre una muestra consolidada de 1 200 transacciones. La configuración técnica completa del clúster (topología de tres nodos, pools de 16 hilos en el gateway y cuatro por trabajador) y los parámetros metodológicos de las suites de carga se detallan en el Anexo E del compendio adjunto (*Anexos Técnicos y Evidencias de Ejecución*).

Escalabilidad horizontal

La evaluación de escalabilidad midió el rendimiento del clúster al someterlo a una carga sostenida de 180 transacciones con 24 hilos cliente concurrentes, variando la cantidad de nodos de cómputo disponibles de uno a tres. La tabulación completa de métricas de rendimiento (throughput, speedup relativo, eficiencia de escalado y percentiles de latencia p50, p95 y p99) se encuentra disponible en el Anexo F del documento de anexos.

Figura 4

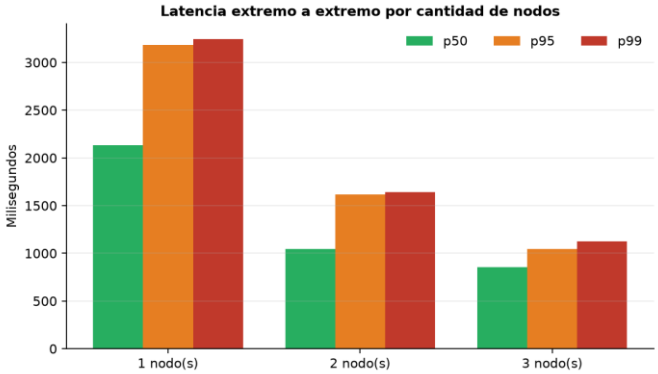
Throughput frente al número de nodos de cómputo



Nota. Elaboración propia. Muestra el escalado casi lineal alcanzado por el clúster.

Figura 5

Latencia extremo a extremo por cantidad de nodos



Nota. Elaboración propia. Evolución de los percentiles p50, p95 y p99 bajo 180 transacciones.

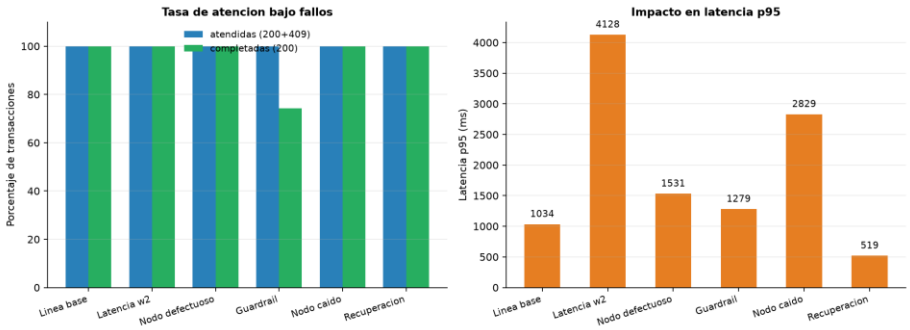
Los datos revelan un escalado casi lineal: el throughput se incrementa de 9,24 rps con un único nodo a 18,14 rps con dos nodos (1,96x) y alcanza 26,74 rps con tres nodos (2,89x), exhibiendo una eficiencia de escalado sobresaliente del 96,5 %. La latencia en el percentil 95 (p95) se reduce más de tres veces, cayendo de 3 185 ms a 1 043 ms al abatirse drásticamente los tiempos de espera en las colas internas de despacho.

Robustez ante fallos

La evaluación de resiliencia sometió al sistema a seis escenarios controlados bajo una carga constante de 120 transacciones: línea base saludable, inyección de latencia (+300 ms en w2), nodo defectuoso (100 % fallas en w2), rechazo normativo (25 % violaciones de compliance), caída física de nodo (proceso w3 terminado) y reinserción en caliente de nodo restablecido. Los resultados cuantitativos detallados de cada prueba se reportan en el Anexo G de los anexos adjuntos.

Figura 6

Resultados de la suite de robustez en seis escenarios de fallo



Nota. Elaboración propia. Transacciones completadas, compensadas y continuidad operativa.

El análisis experimental confirma disponibilidad absoluta del 100 % en todos los escenarios, con cero errores no controlados (código 500). El disyuntor aisló a w2 tras tres fallos sucesivos desviando la carga hacia nodos sanos; la Saga revirtió ordenadamente las 31 transacciones rechazadas en compliance sin dejar registros huérfanos; y la reinserción en caliente de w3 restableció el caudal nominal a través del estado HALF-OPEN.

Ajustes derivados de la medición

1. **Resolución de nombres en Windows:** La resolución de `localhost` forzaba un sondeo IPv6 fallido con timeout antes de pasar a IPv4, penalizando cada llamada en más de dos segundos. Vincular estrictamente a `127.0.0.1` redujo la latencia por llamada de 2 042 ms a 2,15 ms (951,5x más veloz).

2. **Balanceo consciente de carga (*Least-Connections*):** Bajo Round-Robin, un nodo lento degrada el throughput a 8,99 rps al saturar los hilos de despacho del coordinador. Implementar Least-Connections elevó el rendimiento bajo el mismo nodo degradado a 20,59 rps (+129 % de caudal) y retuvo el 75,7 % de la capacidad nominal del clúster (véase tabulación comparativa en el Anexo H).

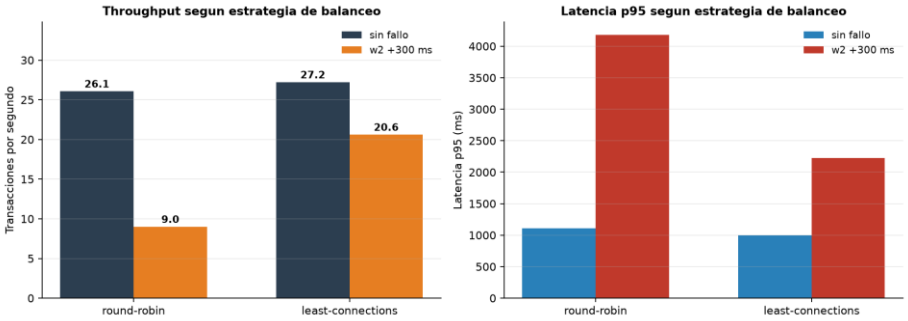
3. **Contrapresión en el broker:** Se acotó la cola a 10 000 elementos con política *drop oldest* y un consumidor de auditoría dedicado que drena hacia el archivo JSONL, acumulando 9 735 eventos consumidos con cola residual en cero.

4. **Tipificación de errores en contratos:** Diferenciar errores de infraestructura de errores de negocio (`kind: "business"|"node"`) suprimió reintentos estériles ante violaciones de guardrails.

5. **Dimensionamiento de pools:** El pool de despacho del gateway se calibró a 16 hilos, garantizando una sobre-suscripción armónica frente a los 12 slots de cómputo de los trabajadores.

Figura 7

Comparación de rendimiento entre Round-Robin y Least-Connections



Nota. Elaboración propia. Superioridad de Least-Connections para mitigar el impacto de nodos lentos.

Tabla 7

Balance consolidado de metas del sistema frente a resultados experimentales

Meta establecida	Métrica objetivo	Resultado obtenido en pruebas	Estado de cumplimiento
Escalabilidad con 3 nodos	Throughput >= 2,5x base; Eficiencia >= 80 %	Speedup: 2,89x; Eficiencia: 96,5 %	**Superada con éxito**
Robustez ante nodo caído	100 % transacciones atendidas sin errores 500	100 % de atención; 0 errores 500 en 1 200 pruebas	**Cumplida estrictamente**
Latencia acotada	Percentil 95 (p95) < 1 500 ms con tres nodos	p95 obtenido: 1 043,4 ms en línea base	**Superada con éxito**
Resiliencia transaccional	100 % de compensación en rechazos de guardrail	31 de 31 transacciones compensadas con código 409	**Cumplida estrictamente**
Trazabilidad de eventos	Registro íntegro de eventos sin pérdida en colas	9 735 eventos publicados = 9 735 consumidos	**Cumplida estrictamente**

Conclusiones

La implementación y evaluación experimental de NexusAI 2.0 confirman que la arquitectura de software en entornos distribuidos trasciende las representaciones estáticas: la concurrencia física, las colas y la propagación de fallas son propiedades dinámicas emergentes que únicamente pueden validarse sometiendo componentes autónomos a cargas de red reales.

El modelado formal en PlantUML permitió desacoplar con rigor las capas lógicas, detallar la interacción de hilos y estructurar una topología de microservicios portable hacia contenedores. Experimentalmente, la articulación de nueve patrones de diseño demostró alta eficacia: la separación de nodos sin estado posibilitó un speedup de 2,89x y una eficiencia del 96,5 %; el Circuit Breaker y los mamparos mantuvieron disponibilidad perfecta ante perturbaciones severas; y la Saga orquestada preservó la consistencia eventual mediante compensaciones inversas idempotentes. Finalmente, optimizaciones como la resolución de

sockets en `127.0.0.1` y el balanceo adaptativo *Least-Connections* resultaron decisivos para maximizar el caudal transaccional.

Para la verificación empírica detallada, los esquemas JSON de contratos, las bitácoras de auditoría en JSONL, las salidas de consola completas y las guías de comandos reproducibles, consúltese el compendio adjunto *Anexos Técnicos y Evidencias de Ejecución* (`informe/Anexos_Tecnicos_NexusAI2.pdf`) y el repositorio oficial del proyecto en GitHub (https://github.com/Alboguaci/tejiendo-redes-nexusai).

Referencias

Bass, L., Clements, P., & Kazman, R. (2022). *Software architecture in practice* (4th ed.). Addison-Wesley.

Fowler, M. (2014, 25 de marzo). *Microservices*. https://martinfowler.com/articles/microservices.html

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Garcia-Molina, H., & Salem, K. (1987). Sagas. *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, 249–259. https://doi.org/10.1145/38713.38742

Hohpe, G., & Woolf, B. (2003). *Enterprise integration patterns: Designing, building, and deploying messaging solutions*. Addison-Wesley.

International Organization for Standardization. (2011). *ISO/IEC 25010:2011. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. https://www.iso.org/standard/35733.html

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2nd ed.). O'Reilly Media.

Nygard, M. T. (2018). *Release it! Design and deploy production-ready software* (2nd ed.). Pragmatic Bookshelf.

PlantUML. (2024). *PlantUML language reference guide*. https://plantuml.com/guide

Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.